

AULA 3

ANÁLISE E
DESENVOLVIMENTO DE
SISTEMAS

linguagem de programação

Lógica e Decisão

Expressões Relacionais, Álgebra Booleana e Controle de Fluxo Condicional



Prof. Alexandre de Souza Fernandes



Agenda

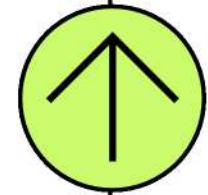
AULA 3

- Variáveis e constantes
- Valores de entrada e saída
- Operadores relacionais e lógicos:
 - Definição, valores booleanos (True/False)
 - Operadores relacionais (`==`, `!=`, `>`, `<`, `>=`, `<=`)
 - Operadores lógicos (*and*, *or* e *not*)
- Estruturas Condicionais Simples e Compostas
 - Estrutura ***if*** e ***else***:
 - Sintaxe
 - Instruções e regras
 - Decisões multiníveis
 - Otimização de fluxo
- Resolução de problemas

**MEDIDAS
PRECISAM**



**SER
TOMADAS**



Situação-problema

AULA 3

Uma equipe foi contratada pelo TRE para projetar o núcleo do motor de validação de dados de uma nova plataforma de auditoria e apoio à votação eletrônica.

DESAFIOS DA SOLUÇÃO

- Modelar as regras acima utilizando:
 - expressões e estruturas condicionais otimizadas.
 - o algoritmo não realize verificações redundantes.
 - não gaste ciclos de CPU desnecessários.

CRITÉRIOS:

• Critério 1 (Elegibilidade ao Voto):

- Inapto: Menor de 16 anos.
- Facultativo: 16 ou 17 anos, 70 anos ou mais, ou analfabeto.
- Obrigatório: Entre 18 e 69 anos e alfabetizado.



Situação-problema

AULA 3

Uma equipe foi contratada pelo TRE para projetar o núcleo do motor de validação de dados de uma nova plataforma de auditoria e apoio à votação eletrônica.

DESAFIOS DA SOLUÇÃO

- Modelar as regras acima utilizando:
 - expressões e estruturas condicionais otimizadas.
 - o algoritmo não realize verificações redundantes.
 - não gaste ciclos de CPU desnecessários.

CRITÉRIOS:

- **Critério 2 (Regra de 2º Turno para Prefeituras):**

- Ocorre apenas em municípios com mais de 200.000 eleitores quando nenhum candidato atinge maioria absoluta ($> 50\%$ dos votos válidos).



Situação-problema

AULA 3

Uma equipe foi contratada pelo TRE para projetar o núcleo do motor de validação de dados de uma nova plataforma de auditoria e apoio à votação eletrônica.

DESAFIOS DA SOLUÇÃO

- Modelar as regras acima utilizando:
 - expressões e estruturas condicionais otimizadas.
 - o algoritmo não realize verificações redundantes.
 - não gaste ciclos de CPU desnecessários.

CRITÉRIOS:

- **Critério 3 (Desempate Técnico e Ordenação de Apuração):**
 - Havendo empate entre candidatos líderes, o sistema deve aplicar critérios estritos de ordenação sem falhas de execução.



variáveis



Uma **variável** é um espaço reservado na memória do computador utilizado para:

- **Armazenar dados e informações** de forma temporária durante a execução de um programa.
- **Permitir** a consulta, a alteração e a manipulação desses valores a qualquer momento.
- **Funcionar** como um compartimento etiquetado que guarda dados essenciais para o funcionamento do sistema.

```
var  
nome_da_variavel : tipo_da_variavel
```

variáveis: regras de declaração

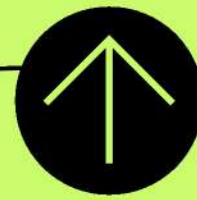


Para que o programa funcione corretamente, a nomeação das variáveis deve seguir regras estritas:

- Não começar com **números**.
- Não utilizar espaços em **branco** ou caracteres **especiais** (com exceção do sublinhado _).
- Não utilizar **palavras reservadas** (keywords) da linguagem de programação (usar *if*, *else* ou *True* como nome de variável).

```
1nota, 2_aluno  
nome completo, valor total  
salario$, e-mail, preço@
```

variáveis: tipos



Tipo da Variável	Código (Python)	Exemplo Prático
Inteiro	int	<code>quantidade_produtos = 50</code>
Ponto Flutuante	float	<code>preco_unitario = 25.99</code>
Texto (String)	str	<code>vila_nascimento = "Barrinha"</code>
Booleano	bool	<code>usuario_logado = True</code>

constante



Uma **constante** é um espaço reservado na memória do computador utilizado para:

- Armazenar dados e informações cujos valores permanecem **fixos**.
- Representar configurações globais, parâmetros fixos (como a gravidade ou o valor de Pi).
- Garantir a segurança e previsibilidade do código.

```
pi=3.14
```

```
g = 10
```

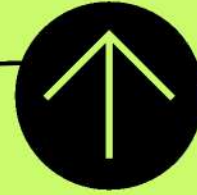
```
NOME_EMPRESA = "Barrinha Solutions"
```



constantes: regras de declaração

Exemplo	Status	Motivo
TAXA_MAXIMA	Certo	Inicia corretamente com letra ou sublinhado.
LIMITE_1	Certo	Contém apenas letras, números e sublinhados após o primeiro caractere.
1FATOR	Errado	Começa com um número, o que gera erro de sintaxe.
TAXA MAXIMA	Errado	Contém espaço em branco.
JUROS%	Errado	Contém caractere especial proibido (%).

constantes: tipos



Tipo da Constante	Código (Python)	Exemplo Prático
Inteiro	int	LIMITE_IDADE = 18
Ponto Flutuante	float	VALOR_PI = 3.14159
Texto (String)	str	NOME_EMPRESA = "Barrinha Solutions"
Booleano	bool	MODULO_MANUTENCAO = False

Comandos de atribuição

AULA 3

COMANDOS DE ENTRADA E SAÍDA

Algoritmo: Apuracao_Eleicoes_Majoritarias

linha 1... Início

linha 2... Leia v_pres

// COMANDO DE ENTRADA

linha 3... Leia v_gov

// COMANDO DE ENTRADA

linha 4... Leia vtotal_validos

// COMANDO DE ENTRADA

linha 5... $\text{perc_pres} \leftarrow (\text{v_pres} / \text{vtotal_validos}) * 100$

linha 6... $\text{perc_gov} \leftarrow (\text{v_gov} / \text{vtotal_validos}) * 100$

linha 7... Escreva ('Perc. Presidencial: ', perc_pres, '%')

// COMANDO DE SAÍDA

linha 8... Escreva ('Perc. Governamental: ', perc_gov, '%')

// COMANDO DE SAÍDA

linha 9 ... Fim.

Comandos de atribuição

AULA 3

COMANDOS DE ENTRADA E SAÍDA

Algoritmo (Python): Apuracao_Eleicoes_Majoritarias

COMANDOS DE ENTRADA

```
v_pres = float(input("Digite a qtde de votos do candidato a Presidente: "))
```

```
v_gov = float(input("Digite a qtde de votos do candidato a Governador: "))
```

```
vtotal_validos = float(input("Digite o total de votos válidos: "))
```

PROCESSAMENTO

```
perc_pres = (v_pres / vtotal_validos) * 100
```

```
perc_gov = (v_gov / vtotal_validos) * 100
```

COMANDOS DE SAÍDA

```
print("Perc. Presidencial: ", perc_pres, "%")
```

```
print("Perc. Governamental: ", perc_gov, "%")
```

Comandos de atribuição

AULA 3

COMANDOS DE ENTRADA E SAÍDA

Algoritmo: soma_de_dois_valores

```
linha 1 ... Algoritmo soma                                     //TÍTULO
linha 2 ... var
linha 3 ...  num1, num2, soma : inteiro                       //DECLARAÇÃO DE VARIÁVEIS
linha 4 ... inicio
linha 5 ...escreva( "Digite o primeiro número")              // COMANDO DE SAÍDA
linha 6 ...leia(num1)                                         // COMANDO DE ENTRADA
linha 7 ...escreva( "Digite o segundo número")               // COMANDO DE SAÍDA
linha 8 ...leia(num2)                                         // COMANDO DE ENTRADA
linha 9 ...  soma ← num1 + num2
linha 10...escreva("A soma dos números digitados é: ", soma) // COMANDO DE SAÍDA
linha 11 ... fim.
```

Comandos de atribuição

AULA 3

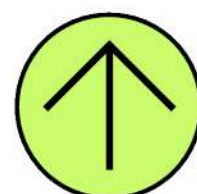
COMANDOS DE ENTRADA E SAÍDA

Algoritmo (Python): soma_de_dois_valores

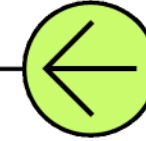
```
# Algoritmo soma
num1 = int(input("Digite o primeiro número: "))
num2 = int(input("Digite o segundo número: "))

soma = num1 + num2

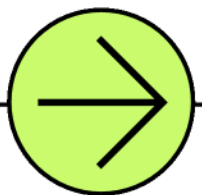
print("A soma dos números digitados é:", soma)
```



Operadores aritméticos



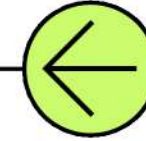
- Constantes e variáveis são **expressões** (numéricas, algébricas, etc.).
- Uma expressão também pode ser um conjunto de operações:
 - aritméticas, lógicas ou relacionais;
 - utilizadas para fazer “cálculos” sobre os valores das variáveis.



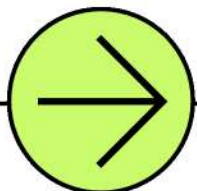
EXEMPLOS

$a + b$; $a - b$; $a * b$; a / b

Operadores aritméticos

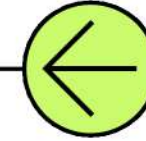


Símbolo	Descrição	Operação / Exemplo (A = 10, B = 3)	Resultado
+	Adição	$A + B$	13
-	Subtração	$A - B$	7
*	Multiplicação	$A * B$	30
/	Divisão Real	A / B	3.333
//	Divisão Inteira	$A // B$	3
%	Resto da Divisão	$A \% B$	1
**	Potenciação	$A ** B$	1000

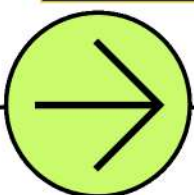


```
a = 10
b = 3
soma = a + b
print("Soma:", soma)
```

Operadores aritméticos

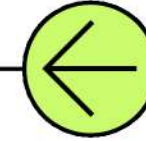


Símbolo	Descrição	Operação / Exemplo (A = 10, B = 3)	Resultado
+	Adição	$A + B$	13
-	Subtração	$A - B$	7
*	Multiplicação	$A * B$	30
/	Divisão Real	A / B	3.333
//	Divisão Inteira	$A // B$	3
%	Resto da Divisão	$A \% B$	1
**	Potênciação	$A ** B$	1000

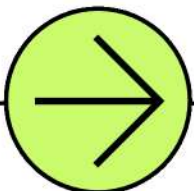


```
a = 10
b = 3
subtracao = a - b
print("Subtração:", subtracao)
```

Operadores aritméticos

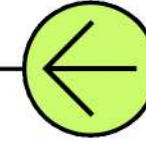


Símbolo	Descrição	Operação / Exemplo (A = 10, B =	Resultado
+	Adição	A + B	13
-	Subtração	A - B	7
*	Multiplicação	A * B	30
/	Divisão Real	A / B	3.333
//	Divisão Inteira	A // B	3
%	Resto da Divisão	A % B	1
**	Potênciação	A ** B	1000

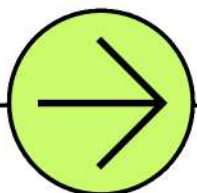


```
a = 10
b = 3
multiplicacao = a * b
print("Multiplicação:", multiplicacao)
```

Operadores aritméticos

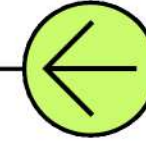


Símbolo	Descrição	Operação / Exemplo (A = 10, B =	Resultado
+	Adição	A + B	13
-	Subtração	A - B	7
*	Multiplicação	A * B	30
/	Divisão Real	A / B	3.333
//	Divisão Inteira	A // B	3
%	Resto da Divisão	A % B	1
**	Potênciação	A ** B	1000

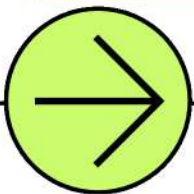


```
a = 10
b = 3
divisao_flutuante = a / b
print("Divisão Ponto Flutuante:", divisao_flutuante)
```

Operadores aritméticos

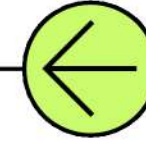


Símbolo	Descrição	Operação / Exemplo (A = 10, B = 3)	Resultado
+	Adição	$A + B$	13
-	Subtração	$A - B$	7
*	Multiplicação	$A * B$	30
/	Divisão Real	A / B	3.333
//	Divisão Inteira	$A // B$	3
%	Resto da Divisão	$A \% B$	1
**	Potênciação	$A ** B$	1000

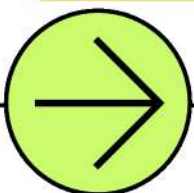


```
a = 10  
b = 3  
print("%.Nf" %(a/b))
```

Operadores aritméticos

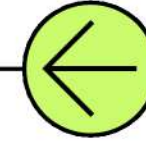


Símbolo	Descrição	Operação / Exemplo (A = 10, B = 3)	Resultado
+	Adição	$A + B$	13
-	Subtração	$A - B$	7
*	Multiplicação	$A * B$	30
/	Divisão Real	A / B	3.333
//	Divisão Inteira	$A // B$	3
%	Resto da Divisão	$A \% B$	1
**	Potênciação	$A ** B$	1000

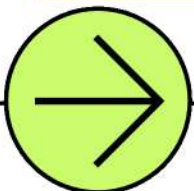


```
a = 10
b = 3
divisao_inteira = a // b
print("Divisão Inteira:", divisao_inteira)
```

Operadores aritméticos

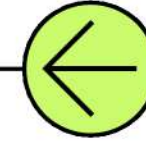


Símbolo	Descrição	Operação / Exemplo (A = 10, B = 3)	Resultado
+	Adição	$A + B$	13
-	Subtração	$A - B$	7
*	Multiplicação	$A * B$	30
/	Divisão Real	A / B	3.333
//	Divisão Inteira	$A // B$	3
%	Resto da Divisão	$A \% B$	1
**	Potênciação	$A ** B$	1000

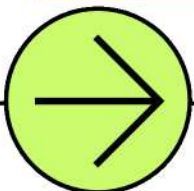


```
a = 10
b = 3
resto = a % b
print("Resto da Divisão:", resto)
```

Operadores aritméticos



Símbolo	Descrição	Operação / Exemplo (A = 10, B = 3)	Resultado
+	Adição	$A + B$	13
-	Subtração	$A - B$	7
*	Multiplicação	$A * B$	30
/	Divisão Real	A / B	3.333
//	Divisão Inteira	$A // B$	3
%	Resto da Divisão	$A \% B$	1
**	Potênciação	$A ** B$	1000



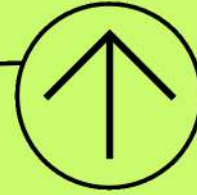
```
a = 10
b = 3
potencia = a**b
print("Potenciação:", potencia)
```

expressões complexas e ordem

- É possível compor expressões complexas como por exemplo: **$a = b * ((2 / c) + (9 \% d * 8))$** .
- Em Python, os operadores são avaliados na seguinte ordem:
 - **`**`**
 - **`*`, `/`, `//`**, na ordem em aparecerem na expressão
 - **`%`**
 - **`+`, `-`**, na ordem em aparecerem na expressão

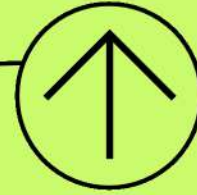
```
>>> 5 + 2**3 * 4 % 3 - 1
>>> 6
```

Precedência



- Operadores com a mesma precedência são executados da esquerda para a direita.
 - $6-3+2$ é igual a ?
 - ✓ ■ $(\rightarrow)$: $6-3$ é igual a 3, $3+2$ é igual a 5
 - $(\leftarrow)$: $3+2$ é igual a 5, $6-5$ é igual a 1
- **Exceção:** operador potencia $**$.
 - $2 ** 3 ** 2$ é igual a ?
 - ✓ ■ $(\rightarrow)$: $2 ** 3$ é igual a 8, $8 ** 2$ é igual a 64
 - $(\leftarrow)$: $3 ** 2$ é igual a 9, $2 ** 9$ é igual a 512

Precedência



- Para determinar a ordem, use parênteses o quão necessário for para deixar claro em qual ordem a expressão é avaliada

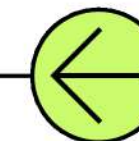
```
>>> 5 + 10 % 3
```

```
>>> 6
```

```
>>> (5 + 10) % 3
```

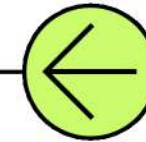
```
>>> 0
```

Operadores relacionais



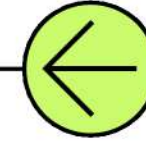
Símbolo	Descrição	Operação / Exemplo (A = 10, B = 3)	Resultado
"="	Igualdade	A = B (10 é igual a 3?)	FALSO
!="	Diferença	A != B (10 é diferente de 3?)	VERDADEIRO
>	Maior que	A > B (10 é maior que 3?)	VERDADEIRO
<	Menor que	A < B (10 é menor que 3?)	FALSO
>=	Maior ou igual a	A >= B (10 é maior ou igual a 3?)	VERDADEIRO
<=	Menor ou igual a	A <= B (10 é menor ou igual a 3?)	FALSO

Operadores lógicos



Comando	Descrição	Operação Exemplo (A = True, B = False)	Resultado
and	E (Conjunção)	A and B (True se ambos forem True)	FALSO
or	OU (Disjunção)	A or B (True se pelo menos um for True)	VERDADEIRO
not	NÃO (Negação)	not A (Inverte o estado lógico de A)	FALSO

Expressão *and*



- 'expressão' **and** 'expressão': Só é V se TUDO for Verdade.
 - Sua **tabela-verdade** é:

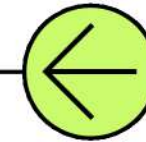
Op1	Op2	Op1 and Op2
V	V	V
V	F	F
F	V	F
F	F	F

p	q	p$\wedge$q
V	V	V
V	F	F
F	V	F
F	F	F

- Resultado lógico:

```
>>> a = 0
>>> b = 1
>>> ( a == 0 and b == 0 )
False
```

Expressão *or*



- 'expressão' **or** 'expressão': Só é F se TUDO for Falso..
 - Sua **tabela-verdade** é:

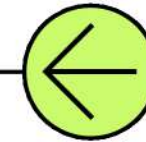
Op1	Op2	Op1 or Op2
V	V	V
V	F	V
F	V	V
F	F	F

p	q	p $\vee$ q
V	V	V
V	F	V
F	V	V
F	F	F

- Resultado lógico:

```
>>> a = 0
>>> b = 1
>>> ( a == 0 or b == 0 )
True
```

Expressão *not*



- '**not** expressão': Retorna V quando a expressão é Falsa e vice-versa.
 - Sua **tabela-verdade** é:

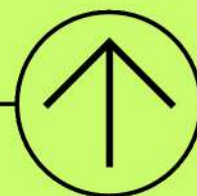
Op1	not Op1
V	F
F	V

p	~ p
V	F
F	V

- Resultado lógico:

```
>>> a = 0
>>> b = 1
>>> not ( a != b )
False
```

Exemplo



Se **$x = 6$** , o que será impresso pelo programa, dadas as condições abaixo?

- a)** $0 < x < 5$ **b)** $x > 0$ or $x < 5$ **c)** $x > 0$ and $x < 5$ **d)** $x > 0$ and < 5

Exemplo



Se $x = 6$, o que será impresso pelo programa, dadas as condições abaixo?

a) $0 < x < 5$

```
>>> x = 6
>>> 0 < x < 5
```

b) $x > 0$ or $x < 5$

```
>>> x = 6
>>> x > 0 or x < 5
True
```

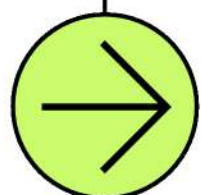
c) $x > 0$ and $x < 5$

```
>>> x = 6
>>> x > 0 and x < 5
False
```

d) $x > 0$ and < 5

```
x = 6
>>> x > 0 and < 5
  File "<stdin>",
    line 1
    x > 0 and < 5
      ^
```

Expressões lógicas

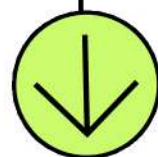


Expressões lógicas são expressões formadas a partir do uso de

- variáveis e constantes;
- operadores aritméticos;
- operadores relacionais
- operadores lógicos.

As expressões lógicas são avaliadas e retornam sempre

- um valor lógico:
 - Verdadeiro (True)
 - Falso (False).



EXEMPLO

Algoritmo: validacao_eleicao_presidencial

Lin	Algoritmo	Cmd/Op
1	Início	Início
2	Leia v_cand1	Entrada
3	Leia v_cand2	Entrada
4	Leia t_eleit_aptos	Entrada
5	$v_val \leftarrow v_cand1 + v_cand2$	Op. arit.
6	$perc_cand1 \leftarrow (v_cand1 / v_val) * 100$	Op. arit.
7	$maioria_abs \leftarrow perc_cand1 > 50.0$	Op. rel.
8	$gr_coleg \leftarrow t_ eleit_aptos \geq 200000$	Op. rel.
9	$eleito_1turno \leftarrow maioria_abs \text{ ou } (n\grave{a}o \text{ gr_coleg})$	Op. lóg.
10	$seg_turno \leftarrow (n\grave{a}o \text{ maioria_abs}) \text{ e } gr_coleg$	Op. lóg.
11	Escreva ('Perc. do Candidato 1: ', perc_cand1, '%')	Saída
12	Escreva ('Eleito no 1º Turno: ', eleito_1turno)	Saída
13	Escreva ('Haverá 2º Turno: ', seg_turno)	Saída
14	Fim	Fim

Expressões lógicas

Expressões lógicas são expressões formadas a partir do uso de

- variáveis e constantes;
- operadores aritméticos;
- operadores relacionais
- operadores lógicos.

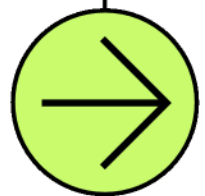
As expressões lógicas são avaliadas e retornam sempre

- um valor lógico:
 - Verdadeiro (True)
 - Falso (False).

EXEMPLO

Algoritmo (Python): validacao_eleicao_presidencial

```
# ENTRADA
v_cand1 = int(input("Digite os votos do candidato 1: "))
v_cand2 = int(input("Digite os votos do candidato 2: "))
t_eleit_apos = int(input("Digite o total de eleitores aptos: "))
# OPERAÇÕES ARITMÉTICAS
v_val = v_cand1 + v_cand2
perc_cand1 = (v_cand1 / v_val) * 100
# OPERAÇÕES RELACIONAIS
maioria_abs = perc_cand1 > 50.0
gr_coleg = t_eleit_apos >= 20000
# OPERAÇÕES LÓGICAS
eleito_1turno = maioria_abs or (not gr_coleg)
seg_turno = (not maioria_abs) and gr_coleg
# SAÍDA
print("Perc. do Candidato 1: ", perc_cand1, "%")
print("Eleito no 1º Turno: ", eleito_1turno)
print("Haverá 2º Turno: ", seg_turno)
```

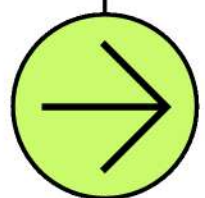


ESTRUTURA SE ...ENTÃO

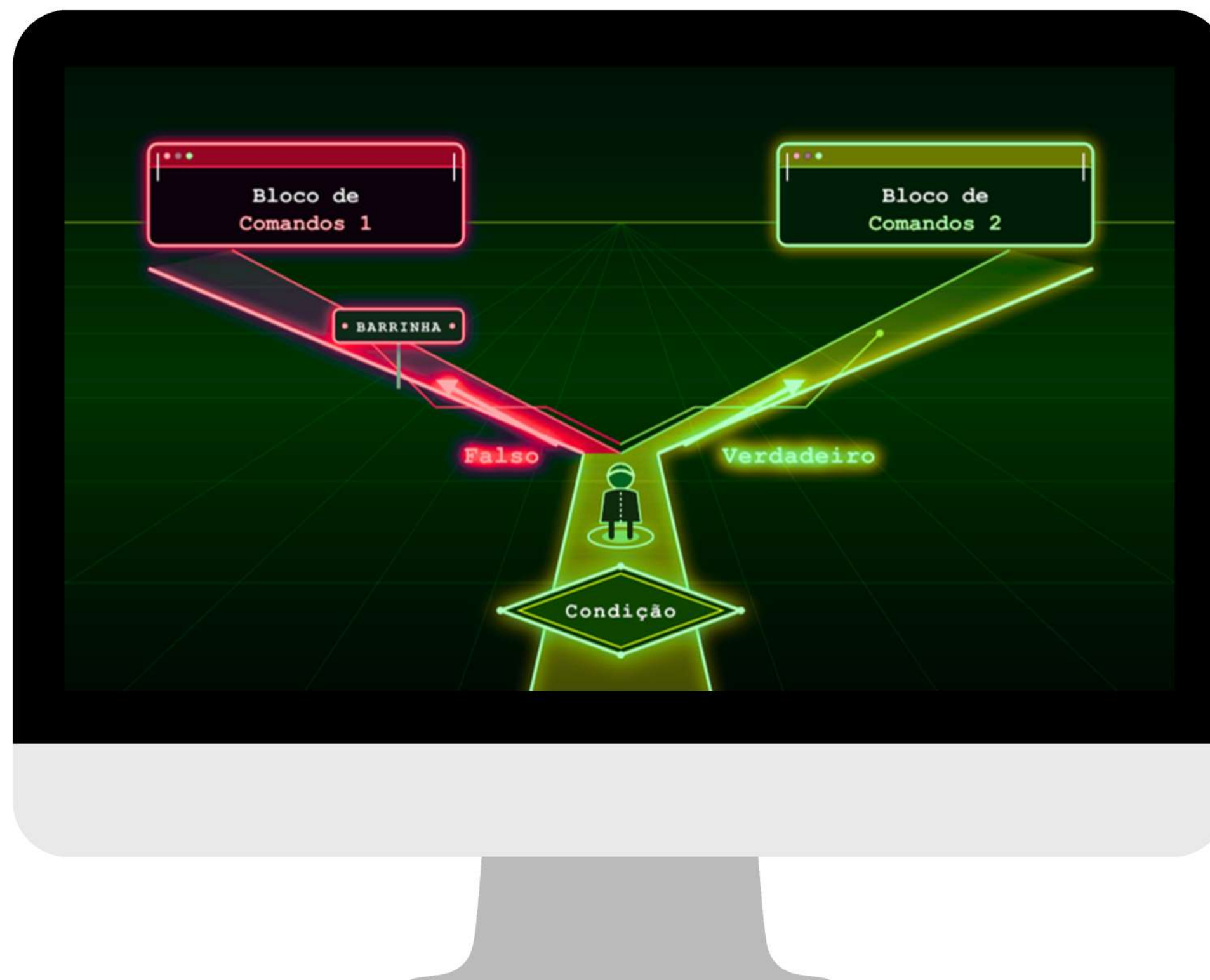
A estrutura de decisão permite que o programa avalie uma **condição** e tome caminhos diferentes dependendo do resultado.

- Ela analisa uma **expressão lógica** ou **relacional** que obrigatoriamente resulta em **Verdadeiro** ou **Falso**.
- **SE** a condição for avaliada como **verdadeira**, **ENTÃO** o programa executa um **bloco específico de instruções**.
- **SENÃO**, ela define uma rota alternativa de ações caso a condição inicial seja **falsa**.

Estrutura de decisão



ESTRUTURA SE ...ENTÃO



Exemplo



Algoritmo: verificar_numero_par

Linha	Algoritmo	Cmd/Op
1	Início	Início
2	Leia numero	Entrada
3	Se numero % 2 == 0 então	Condicional (Se)
4	Escreva ('O número digitado é par.')	Saída
5	Senão	Condicional (Senão)
6	Escreva ('O número digitado é ímpar.')	Saída
7	FimSe	Fim Condicional
8	Fim	Fim

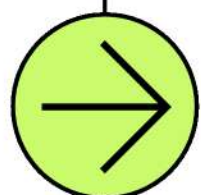
Exemplo



Algoritmo (Python): verificar_numero_par

```
#PROGRAMA PAR
numero = int(input())
if numero % 2 == 0:
    print("O número digitado é par.")
else:
    print("O número digitado é ímpar.")
```

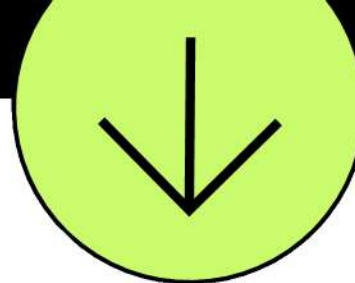
Estrutura de decisão



ESTRUTURA SE ...ENTÃO

A estrutura de decisão permite que o programa avalie uma **condição** e tome caminhos diferentes dependendo do resultado.

- Ela analisa uma **expressão lógica** ou **relacional** que obrigatoriamente resulta em **Verdadeiro** ou **Falso**.
- **SE** a condição for avaliada como **verdadeira**, **ENTÃO** o programa executa um **bloco específico de instruções**.
- **SENÃO**, ela define uma rota alternativa de ações caso a condição inicial seja **falsa**.

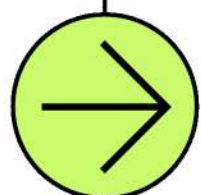


EXEMPLO

Algoritmo: validacao_eleicao_presidencial

Lin	Algoritmo	Cmd/Op
1	Início	Início
2	Leia v_cand1	Entrada
3	Leia v_cand2	Entrada
4	$v_val \leftarrow v_cand1 + v_cand2$	Op. arit.
5	$perc_cand1 \leftarrow (v_cand1 / v_val) * 100$	Op. arit.
6	Se perc_cand1 > 50.0 então	Condicional (Se)
7	Escreva ('Eleito no 1º Turno com ', perc_cand1, '%')	Saída
8	Senão	Condicional (Senão)
9	Escreva ('Não eleito no 1º Turno. Perc: ', perc_cand1, '%')	Saída
10	FimSe	Fim Condicional
11	Fim	Fim

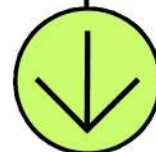
Estrutura de decisão



ESTRUTURA SE ...ENTÃO

A estrutura de decisão permite que o programa avalie uma **condição** e tome caminhos diferentes dependendo do resultado.

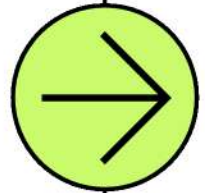
- Ela analisa uma **expressão lógica** ou **relacional** que obrigatoriamente resulta em **Verdadeiro** ou **Falso**.
- **SE** a condição for avaliada como **verdadeira**, **ENTÃO** o programa executa um **bloco específico de instruções**.
- **SENÃO**, ela define uma rota alternativa de ações caso a condição inicial seja **falsa**.



EXEMPLO

Algoritmo (Python): validacao_eleicao_presidencial

```
# ENTRADA
v_cand1 = int(input("Digite os votos do candidato 1:
"))
v_cand2 = int(input("Digite os votos do candidato 2:
"))
# PROCESSAMENTO ARITMÉTICO
v_val = v_cand1 + v_cand2
perc_cand1 = (v_cand1 / v_val) * 100
# ESTRUTURA CONDICIONAL SIMPLES (IF / ELSE)
if perc_cand1 > 50.0:
    print("Eleito no 1º Turno com ", perc_cand1,
"%")
else:
    print("Não eleito no 1º Turno. Perc: ",
perc_cand1, "%")
```



DECISÕES MULTINÍVEL

- Se o comando if implica uma condição, tal condição pode haver dentro dela outra(s) condições.
- Em outras palavras, podemos aplicar o comando if dentro de outros if's.



Exemplo



No Python, usando apenas operadores relacionais e aritméticos, vamos escrever um programa que lê um número e verifica em qual dos seguintes casos o número se enquadra:

- a)** Par e menor que 100
- b)** Par e maior ou igual a 100
- c)** Ímpar e menor que 100
- d)** Ímpar e maior ou igual a 100

Exemplo

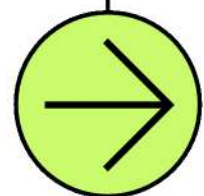


No Python, usando apenas operadores relacionais e aritméticos, vamos escrever um programa que lê um número e verifica em qual dos seguintes casos o número se enquadra:

- a)** Par e menor que 100
- b)** Par e maior ou igual a 100
- c)** Ímpar e menor que 100
- d)** Ímpar e maior ou igual a 100

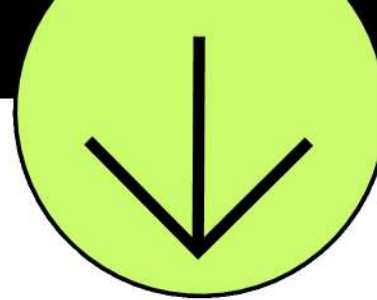
```
numero = int(input("Digite um número: "))
if (numero % 2 == 0): # se o número for par
    if (numero < 100):
        print("O número é par e menor que 100")
    else:
        print("O número é par e maior ou igual que 100")
else: # se o número for ímpar
    if (numero < 100):
        print("O número é ímpar e menor que 100")
    else:
        print("O número é ímpar e maior ou igual que 100")
```

Estrutura de decisão



DECISÕES MULTINÍVEL

- Se o comando if implica uma condição, tal condição pode haver dentro dela outra(s) condições.
- Em outras palavras, podemos aplicar o comando if dentro de outros if's.

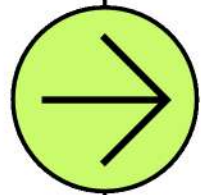


EXEMPLO

Algoritmo (Python): validacao_eleicao_presidencial

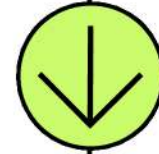
Lin	Algoritmo	Comando/Operação
1	Início	Início
2	Leia v_cand1	Entrada
3	Leia v_cand2	Entrada
4	Leia t_eleit_aptos	Entrada
5	$v_val \leftarrow v_cand1 + v_cand2$	Op. arit.
6	$perc_cand1 \leftarrow (v_cand1 / v_val) * 100$	Op. arit.
7	$gr_coleg \leftarrow t_eleit_aptos \geq 200000$	Op. rel.
8	Se gr_coleg então	Condicional (Se)
9	Se perc_cand1 > 50.0 então	Condicional (Se)
10	Escreva ('Eleito no 1º Turno com ', perc_cand1, '%')	Saída
11	Senão	Condicional (Senão)
12	Escreva ('Haverá 2º Turno. Perc: ', perc_cand1, '%')	Saída
13	FimSe	Fim Condicional
14	Senão	Condicional (Senão)
15	Se v_cand1 > v_cand2 então	Condicional (Se)
16	Escreva ('Eleito no 1º Turno (Maioria Simples)')	Saída
17	Senão	Condicional (Senão)
18	Escreva ('Não eleito no 1º Turno')	Saída
19	FimSe	Fim Condicional
20	FimSe	Fim Condicional
21	Fim	Fim

Estrutura de decisão



DECISÕES MULTINÍVEL

- Se o comando if implica uma condição, tal condição pode haver dentro dela outra(s) condições.
- Em outras palavras, podemos aplicar o comando if dentro de outros if's.



EXEMPLO

Algoritmo (Python): validacao_eleicao_presidencial

```
# ENTRADA
v_cand1 = int(input("Digite os votos do candidato 1: "))
v_cand2 = int(input("Digite os votos do candidato 2: "))
t_eleit_aptos = int(input("Digite o total de eleitores aptos: "))
# OPERAÇÕES ARITMÉTICAS E RELACIONAIS
v_val = v_cand1 + v_cand2
perc_cand1 = (v_cand1 / v_val) * 100
gr_coleg = t_eleit_aptos >= 200000
# ESTRUTURA CONDICIONAL IF-ELSE
if gr_coleg:
    if perc_cand1 > 50.0:
        print("Eleito no 1º Turno com ", perc_cand1, "%")
    else:
        print("Haverá 2º Turno. Perc: ", perc_cand1, "%")
else:
    if v_cand1 > v_cand2:
        print("Eleito no 1º Turno (Maioria Simples)")
    else:
        print("Não eleito no 1º Turno")
```

EXERCÍCIO

1

Considere os tipos de dados e operadores em Python. Qual das instruções abaixo resulta em um valor booleano True?

A

```
bool("False") == False
```

B

```
10 >= 5 and 3 != 3
```

C

```
8 > 9 or not (2 > 1)
```

D

```
not (14 > 100) and not (1 > 2)
```

EXERCÍCIO

2

Considere as variáveis
 $x = 5$, $y = 10$ e $z = 2$.

Qual é o resultado
exato da avaliação da
expressão

$x + z * 3 > y$ and not $z == 2$ or
 $y // 2 == 5$
?

A

True

B

False, pois not $z == 2$ avalia para False, anulando toda a expressão devido à precedência do and.

C

Erro de sintaxe em tempo de execução pela falta de parênteses delimitadores entre operações relacionais e aritméticas.

D

False, pois a multiplicação $z * 3$ tem precedência menor que a soma $x + z$.

EXERCÍCIO

3

Analise o trecho de código abaixo:

```
numero = 9
```

```
if numero > 3:
```

```
    if numero < 7:
```

```
        print("Opção A")
```

```
    else:
```

```
        if numero > -10:
```

```
            print("Opção B")
```

```
        else:
```

```
            print("Opção C")
```

```
else:
```

```
    print("Opção D")
```

Qual mensagem será exibida no console?

A

Opção A

B

Opção B

C

Opção C

D

Opção D

EXERCÍCIO

4

Escreva um programa que simula o jogo conhecido como “Pedra, Papel e Tesoura” de um(a) jogador(a) contra o(a) outro(a).

EXERCÍCIO

5

Escreva um programa em Python que receba a idade (int) e a condição de alfabetização (bool ou string "S"/"N") de um cidadão. O programa deve imprimir uma das três classificações oficiais:

1. "Voto Não Permitido" (idade menor que 16 anos).
2. "Voto Facultativo" (16 ou 17 anos, 70 anos ou mais, ou não alfabetizado).
3. "Voto Obrigatório" (entre 18 e 69 anos e alfabetizado).

EXERCÍCIO

6

Desenvolva um algoritmo que:

1. leia o nome e o número de votos de 3 candidatos concorrentes (**cand1**, **cand2**, **cand3**)
2. imprima a classificação do 1º ao 3º colocado em ordem decrescente, utilizando exclusivamente estruturas condicionais aninhadas (if-else), sem o uso de listas ou funções como **sort()**.

DESAFIO

D

No clássico jogo "Pedra, Papel e Tesoura" (abordado na Exercício 4), codificamos as opções como números: Pedra = 0, Papel = 1, Tesoura = 2. A vitória do Jogador 1 sobre o Jogador 2 pode ser expressa sem múltiplos ifs encadeados através da expressão modular:

```
if (jog1 - jog2) % 3 == 1:  
    print("Jogador 1 venceu!")
```

Demonstre algebricamente por que a fórmula $(\text{jog1} - \text{jog2}) \% 3 == 1$ valida todas as 3 situações de vitória do Jogador 1 (Papel vence Pedra, Tesoura vence Papel, Pedra vence Tesoura).

GABARITO

:)

1. `NameError` (A variável `a` não foi inicializada antes de ser usada na linha `a = a + 1`).
2. `ValueError` (O valor armazenado na variável `a` no fim do programa é 8).
3. `TypeError` (Tentativa de somar um número float com uma string).
4. `TypeError` (Tentativa de alterar o valor de um índice em uma tupla, que é um objeto imutável).
5. `ValueError` (Incompatibilidade no desempacotamento: há 5 itens na lista, mas apenas 4 variáveis para recebê-los).
6. (Desafio) Erro de referência de memória (Ambas as variáveis apontam para o mesmo objeto na memória, então alterar uma afeta a outra).